

HDL

└─ Verilog 2001
└─ Verilog 2005

* System Verilog got merged with verilog

* It is an hardware descriptive lang.
& not programming

└─ we are just describing the circuit

Modeling style

└─ Structural representation
└─ Behavioral

① Gate Level modeling.

ie





```

module mux_2x1_gate (
    input
    _____
    _____);

```

```

not    NO (sn, s)
and    AO (go, x0, sn)
and    AI (g1, x1, s)
or     OO (f, go, g1)

```

end module

② Data flow modeling

xNOR

Category	Examples	Bit Length
Bitwise	$\sim A$, $+A$, $-A$ $A \& B$, $A B$, $A \sim \wedge B$, $A \wedge \sim B$	$L(A)$ $\text{MAX } (L(A), L(B))$
Logical	$!A$, $A \&\& B$, $A B$	1 bit
Reduction	$\&A$, $\sim\&A$, $ A$, $\sim A$, $\wedge \sim A$, $\sim \wedge A$	1 bit
Relational	$A == B$, $A != B$, $A > B$, $A < B$ $A >= B$, $A <= B$ $A === B$, $A !== B$	1 bit
Arithmetic	$A + B$, $A - B$, $A * B$, A / B $A \% B$	$\text{MAX } (L(A), L(B))$
Shift	$A \ll B$, $A \gg B$	$L(A)$
Concatenate	$\{A, \dots, B\}$	$L(A) + \dots + L(B)$
Replication	$\{B\{A\}\}$	$B * L(A)$

i.e

assign $f = \sim s \& x_0 \mid s \& x_1$

continuous assignment

if RHS changes LHS also changes

→ so used assign

③ Behavioural Representation.

i.e

output reg f sensitivity list

always @ (—)

if (s)
begin

x_1, x_0, s

$f = x_1$

end

else

begin

$f = x_0$

↳ we should have used assign over here

but not using.
↳ cause they are not continuous assignment — structural, but rather behaviour —

end

at (procedural assign-)
↳ we will use

in recent version, we ^{always}
can put `;` in place of `or`

* if output is inside procedural block
& is changing then need to be of
type "register"

i.e case (s) optional
 0: f = x0;
 1: f = x1;
 default: f = x0
endcase

Take away

① Continuous Assignment (Data flow Modeling)

↳ continuously evaluate RHS whenever
any input changes

↳ used outside procedural blocks
i.e always & initial

↳ for simple combinational logic

② Procedural Assignment (Behavioural Modeling)

↳ execute only when triggered by sensitivity list or control flow

↳ further blocking ($=$) & non-blocking ($\leq$)

↳ for sequential logic or complex combinational logic

How to write good verilog code.

1) Decide the type of hardware you want to use.

- ↳ combinational: • out. dep. on current input
• No memory
• output changes immediately after a small propagation delay.
- ↳ Sequential: • out. dep. on current input + past state.
• Has memory
• State changes at a specific clock edge.

2) To create combinational Logic

- Continuous Assignment (assign)

- ↳ to express single line of code, one equation
- ↳ Directly within module, NOT inside (always) block.
i.e. assign out = a & b | c;

- Procedural Block (always @(*))

- ↳ to implement more complex logic using if, case

→ use blocking ($=$) assignment)

To create
Sequential circuit

- Procedural block (always @(posedge clk))

→ when need to create reg, f.f. or memory element
→ use non-blocking ($\leq$) assignment

③ Avoid errors

- Using Non-Blocking ($\leq$) in always @(*)

→ your simulation will not match the synthesized hardware. ($\leq$) can infer a latch (unintended memory)

- Using Blocking ($=$) in always @(posedge clk)

→ creates a race condition, simulation result will depend on the order of statements, but in real world all f.f. triggers at once

- mixing ($=$) & ($\leq$) in same always block.

→ It leads to unpredictable simulation behavior.

②
Data flow modeling → continuous assignment (assign)
Behavioural modeling → Procedural assignment (always @(*)
always @(—))